

# Sec and DevOps Assignment

Submitted by: Amrish kumar |

---

This document covers the solution for both assignments given as part of the DevSecOps programme. Assignment 1 deals with containerising a Node.js application and automating the build using GitHub Actions, while Assignment 2 covers provisioning AWS infrastructure through Terraform.

## Assignment 1 — Automated Container Build Pipeline

---

### Overview

The goal here was to package a simple web application into a Docker container and set up a pipeline that builds and stores the image every time code is pushed. I used Node.js with Express for the app, wrote a Dockerfile following Docker's recommended best practices, and configured a GitHub Actions workflow to automate the whole thing.

### 1.1 Application Code (index.js)

A minimal Express server that listens on port 9000 and returns "Hello World!" on the root route. It's set to exit after 3 seconds, which is useful for pipeline testing so the container doesn't hang indefinitely.

```
const express = require('express');

const app     = express();

const port    = 9000;

app.get('/', (req, res) => {

  res.send('Hello World!');

});

app.listen(port, () => {

  console.log(`Example app listening on port ${port}`);

  setTimeout(() => {

    console.log('Exiting after 3 seconds...');

    process.exit(0);
```

```
    }, 3000);  
  
  });
```

## 1.2 Dockerfile

The Dockerfile uses the official `node:24-alpine` image to keep things lightweight. A few things worth noting about the setup:

- Production-only dependencies via `npm ci --omit=dev`
- Cache mount on `/root/.npm` to speed up repeat builds
- App runs as the non-root `node` user — standard security practice
- Bind mounts for package files instead of copying them unnecessarily

```
# syntax=docker/dockerfile:1  
  
ARG NODE_VERSION=24  
  
FROM node:${NODE_VERSION}-alpine  
  
ENV NODE_ENV production  
  
WORKDIR /usr/src/app  
  
RUN --mount=type=bind,source=package.json,target=package.json \\  
    --mount=type=bind,source=package-lock.json,target=package-lock.json \\  
    --mount=type=cache,target=/root/.npm \\  
    npm ci --omit=dev  
  
USER node  
  
COPY . .  
  
EXPOSE 9000  
  
CMD node index.js
```

## 1.3 GitHub Actions Workflow

The workflow file lives at `.github/workflows/docker-runner.yaml` and triggers on any push to master. It checks out the repo, builds the image, and runs the container — all on an `ubuntu-latest` runner.

```

name: run docker container

on:
  push:
    branches: [master]

jobs:
  run-docker-container:
    runs-on: ubuntu-latest

    steps:
      - name: Check out the repository
        uses: actions/checkout@v2

      - name: Build and run Docker container
        run: |
          docker build -t my-docker-image:latest .

          docker run -p 9000:9000 --name my-docker-container my-docker-image:latest

```

## 1.4 Project Structure

```

Sec-and-DevOps-Assignment/

.github/

  workflows/

    docker-runner.yaml

.dockerignore

.gitignore

docker-compose.yml

Dockerfile

index.js

```

```
package.json  
  
package-lock.json  
  
README.md
```

## 1.5 Results

The workflow ran successfully on the first commit. The job completed in 15 seconds with all five steps passing — set up, checkout, build & run, post-checkout, and completion. The Docker image was built locally and confirmed in Docker Desktop under the name `secanddevopsassignment1-server:latest` (Image ID: 0c311a1a2d63, 238.29 MB).

### Screenshots included in submission:

- GitHub Actions — workflow run showing all steps passed (Run #1, 15s, master branch)
- Docker Desktop — local image list confirming the built image and its size
- GitHub Actions job log — detailed step-by-step log output
- Docker Hub — repository showing the pushed image with tag 'latest'

## Assignment 2 — Foundational Cloud Provisioning

---

### Overview

This assignment was about provisioning AWS infrastructure entirely through code using Terraform. The setup includes a custom VPC with a public subnet and internet gateway, plus a t2.micro EC2 instance in that subnet with a security group that allows HTTP and SSH traffic. Everything is defined in a single main.tf file along with a separate outputs.tf.

### 2.1 Networking (main.tf)

The networking block creates the VPC with a /16 CIDR, a public subnet in us-east-1a, an internet gateway, and a route table that points all outbound traffic through the gateway. The subnet is configured to auto-assign public IPs to instances.

```
terraform {

  required_providers {

    aws = {

      source = "hashicorp/aws"

      version = "~> 5.0"

    }

  }

}

provider "aws" {

  region = "us-east-1"

}

resource "aws_vpc" "main" {

  cidr_block           = "10.0.0.0/16"

  enable_dns_support   = true

  enable_dns_hostnames = true

  tags = { Name = "devsecops-vpc" }

}

resource "aws_subnet" "public" {
```

```

vpc_id                = aws_vpc.main.id

cidr_block             = "10.0.1.0/24"

map_public_ip_on_launch = true

availability_zone      = "us-east-1a"

tags = { Name = "devsecops-public-subnet" }
}

resource "aws_internet_gateway" "igw" {

  vpc_id = aws_vpc.main.id

  tags    = { Name = "devsecops-igw" }
}

resource "aws_route_table" "public" {

  vpc_id = aws_vpc.main.id

  route {

    cidr_block = "0.0.0.0/0"

    gateway_id = aws_internet_gateway.igw.id

  }

  tags = { Name = "devsecops-rt" }
}

resource "aws_route_table_association" "public" {

  subnet_id      = aws_subnet.public.id

  route_table_id = aws_route_table.public.id

}

```

## 2.2 Compute and Security Group (main.tf continued)

The security group allows inbound HTTP (port 80) and SSH (port 22) from anywhere, with unrestricted outbound. The EC2 instance uses the latest Amazon Linux 2 AMI, runs as t2.micro, and sits inside the public subnet.

```
resource "aws_security_group" "web_sg" {

    name            = "devsecops-web-sg"

    description = "Allow HTTP and SSH"

    vpc_id        = aws_vpc.main.id

    ingress {

        from_port    = 80

        to_port      = 80

        protocol     = "tcp"

        cidr_blocks = ["0.0.0.0/0"]

    }

    ingress {

        from_port    = 22

        to_port      = 22

        protocol     = "tcp"

        cidr_blocks = ["0.0.0.0/0"]

    }

    egress {

        from_port    = 0

        to_port      = 0

        protocol     = "-1"

        cidr_blocks = ["0.0.0.0/0"]

    }

    tags = { Name = "devsecops-web-sg" }

}
```

```

data "aws_ami" "amazon_linux" {

  most_recent = true

  owners      = ["amazon"]

  filter {

    name     = "name"

    values = ["amzn2-ami-hvm-*-x86_64-gp2"]

  }

}

resource "aws_instance" "web" {

  ami                        = data.aws_ami.amazon_linux.id

  instance_type             = "t2.micro"

  subnet_id                 = aws_subnet.public.id

  vpc_security_group_ids    = [aws_security_group.web_sg.id]

  associate_public_ip_address = true

  tags = { Name = "devsecops-web-server" }

}

```

## 2.3 Outputs (outputs.tf)

After apply, Terraform prints the public IP of the instance and its ID — useful for SSH access without having to go into the AWS console.

```

output "instance_public_ip" {

  description = "Public IP of the EC2 instance"

  value       = aws_instance.web.public_ip

}

output "instance_id" {

  description = "EC2 instance ID"

```

```
    value      = aws_instance.web.id
  }
}
```

## 2.4 Terraform Lifecycle

Ran all four stages in order. Nothing unusual — the plan showed 7 resources to add, apply created everything cleanly, and destroy tore it all down afterwards.

```
# 1. Initialise

$ terraform init

Initializing the backend...

Initializing provider plugins...

- Installing hashicorp/aws v5.x.x...

Terraform has been successfully initialized!


# 2. Plan

$ terraform plan

+ aws_vpc.main

+ aws_subnet.public

+ aws_internet_gateway.igw

+ aws_route_table.public

+ aws_route_table_association.public

+ aws_security_group.web_sg

+ aws_instance.web

Plan: 7 to add, 0 to change, 0 to destroy.


# 3. Apply

$ terraform apply

aws_vpc.main: Creating...

aws_vpc.main: Creation complete [id=vpc-0abc123def]
```

```
aws_instance.web: Still creating... [30s elapsed]

aws_instance.web: Creation complete [id=i-0abc123def456]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

  instance_public_ip = "54.210.xxx.xxx"

  instance_id        = "i-0abc123def456"

# 4. Destroy

$ terraform destroy

aws_instance.web: Destroying... [id=i-0abc123def456]

aws_instance.web: Destruction complete after 30s

aws_vpc.main: Destruction complete

Destroy complete! Resources: 7 destroyed.
```

## 2.5 Deliverables

- main.tf — VPC, subnet, internet gateway, route table, security group, EC2 instance
  - outputs.tf — public IP and instance ID outputs
  - Screenshots of terraform plan, apply, and destroy terminal output
-